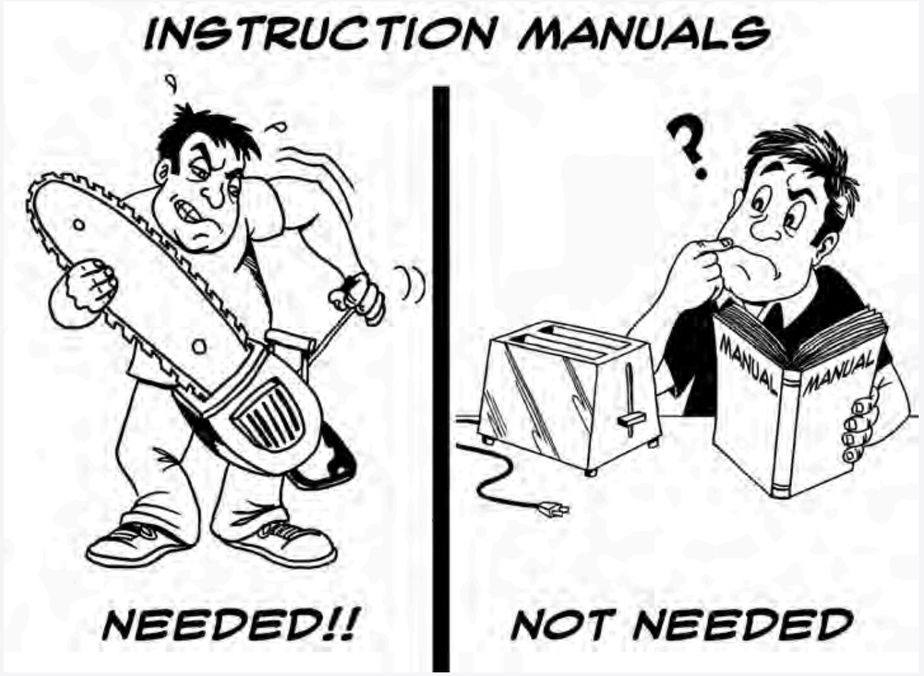
INSTITUTE OF TECHNOLOGY OF CAMBODIA

DEPARTMENT OF APPLIED MATHEMATICS AND STATISTICS

Advanced Programming For Data Science

Knowing What to Comment

The Art of Readable Code



Presenter

Kong Sattha e20210963
Ly Chungheang e20220980
Sor Raksmey e20221466
SROY Liza e20220912

Why We Write Comments

CORE IDEA

Good comments share what the writer knows and the code does not explain quickly.

01

Clear code

Use a clear name first. Add a comment when the code still takes time to understand.

02

Reasons

Explain why you chose a method. Record what you learned from tests.

03

Known problems

Say what the code cannot do yet. Warn about errors, slow steps or risky inputs.

04

Big picture

Explain how the parts fit together. Summarize long blocks so readers can follow the steps.

Comments That Do Not Help

Reading a comment takes time. It should add useful information or make the code easier to understand.

REPEATS THE CODE

```
// The class definition for Account
class Account {
    ...
};
```

The class name already tells us this is an Account. Remove the extra comment. Keep a comment if it makes a difficult line quicker to understand.

A NAME THAT EXPLAINS THE JOB

Unclear: CleanReply(...)

Clear: EnforceLimitsFromRequest(...)

“Clean” does not tell us what changes. The clearer name says that the function limits the reply’s size or item count.

Details a Reader Needs

When nothing is found
Return NULL, which means no matching node.

How far the search goes
depth <= 0 checks only the starting node.

Example with depth = 2
Check the starting node, its children and their children.

```
// Return NULL if no match exists.
// depth <= 0: check the starting node only.
// depth == N: also check N levels below.

Node* FindNodeInSubtree(
    Node* subtree, string name, int depth);
```

These comments help someone choose the inputs and understand the result. The function name alone does not give these details.

Comments can explain return values and special input cases

Why This Choice or Number?

WHY THIS METHOD?

```
// A binary tree was 40% faster than a hash table here.
// Hashing took longer than comparing values.
```

The author tested two ways to store data. This note explains the choice and saves the next reader from running the same test.

WHY THIS FIXED VALUE?

```
image_quality = 0.72;
// Users liked the quality and file size.
```

Explain whether a number came from tests or is only a rough limit. Here, 0.72 is one program’s choice, not a rule for every program.

WHAT TO EXPLAIN

Reason for the choice
Say why you picked this method.

Other choices
Mention what you tried and why you left it.

Known limits
Say which mistakes the code may still make.

Numbers in the code
Explain how you chose a fixed value.

Known Problems and Work to Do

MARKER	WHAT IT MEANS	SHORT EXAMPLE
TODO	Work to do later	// TODO: Add PNG support.
FIXME	A known error to fix	// FIXME: Empty input crashes.
HACK	A quick but messy solution	// HACK: Copy the same fix twice.
XXX	A serious problem to review	// XXX: This may erase saved data.



Questions and Warnings

Why use swap instead of clear()?

```
// Give back this vector’s storage.
std::vector<float>().swap(data);
```

clear() removes the items but may keep the reserved memory. The swap step gives back the vector’s storage.

SURPRISING BEHAVIOR

```
// Calls an outside email service.
// Times out after 1 minute.
void SendEmail(...);
```

SendEmail may wait for another service. In this example, the timeout is one minute.

A web request that waits for this call may feel stuck.

Also warn about changes to data or inputs that make the program very slow.

Comments That Show the Big Picture

SUMMARY INSIDE A FUNCTION

```
def GenerateUserReport():
    # Lock the user's data
    ...
    # Read the user's information
    ...
    # Save the report to a file
    ...
    # Release the lock
```

A reader can scan the steps before reading each line.

Each comment says what the next block is for

File or class
Explain its job and how it fits with other parts.

Connections
Say which parts work together and where reading should start.

Long functions
Add a short comment before each main block.

Separate tasks
Use a named function when a block can stand on its own.

Writing a Clear Comment

A SIMPLE WRITING PROCESS

1. **Write your first thought**
   Do it while the reason is still fresh.

2. **Read it like a new teammate**
   Look for words that do not explain much.

3. **Make the meaning clear**
   Name the input, problem or missing case.

BEFORE

```
// Repeated items make this tricky.
```

AFTER

```
// This code does not handle
// repeated items in the list.
```

What: What does this block do?
Why: Why choose this method?
How: How does this unusual step work?

Key Takeaways

Add useful facts Explain things the code does not show quickly.
Share your reasons Record choices, fixed values and known problems.
Help the next reader Explain surprising steps and the big picture.

Questions & discussion